Numerical Addendum: Dispersion Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

The Dispersion calculator is a cutoff sum of closed-form per-vertex London kernels. The numerical work is the shared ring geometry it consumes, the gates that decide which ring vertices enter the sum, the CHARMM-form switching window whose middle branch matches value and first derivative at the joins, and the shared $T_0/T_1/T_2$ (trace/3, antisymmetric, symmetric-traceless) projection used for output. The listings below are distilled from the source: loops and bookkeeping are trimmed, `cfg(...)` stands for `CalculatorConfig::Get(...)`, and the constants, inequalities, guards, and arithmetic are the implementation's.

1 Ring vertices and gates

`GeometryResult` fills `conf.ring_geometries` before this calculator runs. Each ring geometry carries ordered vertex positions $\boldsymbol{v}_j$, their mean centre $\boldsymbol{c}$, an SVD-fitted unit normal, and a mean centre-to-vertex radius R. The Haigh–Mallion numerical addendum treats the SVD plane fit in full. Dispersion consumes the vertices for the kernel, the centre and radius for acceptance, and the normal only for the shared ring-neighbour coordinate record.

For each atom, the spatial index proposes ring centres within 15.0 Å. That query does not set the dispersion range; the 5.0 Å range is enforced per vertex. The ring-level near-field filter uses `source_extent = 2*geom.radius` and `near_field_exclusion_ratio = 0.5`, so the per-atom path accepts only centre distances greater than R. At equality the ring is omitted. The topology gate then omits the whole ring if the target atom is a ring vertex or is covalently bonded to any vertex.

```
auto nearby_rings =
    spatial.RingsWithinRadius(atom_pos,
        cfg("ring_current_spatial_cutoff"));       // 15.0 A

ctx.distance = (atom_pos - geom.center).norm();
ctx.source_extent = 2.0 * geom.radius;             // ring diameter
if (!filters.AcceptAll(ctx)) continue;             // requires distance > R

if (ring_bonded[ri].count(ai)) continue;           // vertex or bonded neighbour

Mat3 K_ring = Mat3::Zero();
double s_ring = 0.0;
int contacts = 0;

for (size_t vi = 0; vi < ring.atom_indices.size(); ++vi) {
    Vec3 vpos = geom.vertices[vi];
    double r = (atom_pos - vpos).norm();
    auto vertex = ComputeDispVertex(atom_pos, vpos, r);
    if (!vertex.valid) continue;

    K_ring += vertex.K;
    s_ring += vertex.scalar;
    contacts++;
}
```

These gates are omissions. A rejected ring or vertex contributes no tensor, no scalar contact, and no compensating weight elsewhere in the sum.

2 One vertex kernel

For a target atom at $\boldsymbol{r}_i$ and a ring vertex at $\boldsymbol{v}_j$, the helper forms `d_av`, the vector $\boldsymbol{d} = \boldsymbol{r}_i - \boldsymbol{v}_j$ from vertex to atom, and uses the caller's $r = |\boldsymbol{d}|$. The code does not store $\hat{\boldsymbol{d}}$, but the unit direction $\hat{\boldsymbol{d}} = \boldsymbol{d}/r$ is the angular part of the same arithmetic:

$$3\,\frac{d_a d_b}{r^8} - \frac{\delta_{ab}}{r^6} = \frac{1}{r^6}\left(3\hat{d}_a\hat{d}_b - \delta_{ab}\right).$$

Both terms carry units of Å^{-6}; the helper applies $C_6 = 1$.

```
if (r < cfg("singularity_guard_distance")) return result;  // < 0.1 A
if (r > cfg("dispersion_vertex_distance_cutoff")) return result;

double S = DispersionSwitchingFunction(r);
if (S < cfg("dispersion_switching_noise_floor")) return result;

Vec3 d_av = atom_pos - vertex_pos;         // vertex -> atom
double r2 = r * r;
double r6 = r2 * r2 * r2;
double r8 = r6 * r2;

result.scalar = S / r6;

for (int a = 0; a < 3; ++a)
    for (int b = 0; b < 3; ++b) {
        double Iab = (a == b ? 1.0 : 0.0);
        result.K(a,b) = S * (3.0*d_av(a)*d_av(b)/r8 - Iab/r6);
    }

result.valid = true;
```

The tensor part is the symmetric-traceless angular shape $3\hat{\boldsymbol{d}} \otimes \hat{\boldsymbol{d}} - \boldsymbol{I}$, scaled by S/r^6. Along $\hat{\boldsymbol{d}}$ it has eigenvalue $2S/r^6$; in either perpendicular direction it has eigenvalue $-S/r^6$. Those three eigenvalues sum to zero, so each accepted vertex contributes no tensor trace and no antisymmetric part.

The scalar contact stored beside the tensor is the isotropic S/r^6 piece. It is not the trace average of $\boldsymbol{K}$, which is zero by construction. The $-\boldsymbol{I}/r^6$ term inside $\boldsymbol{K}$ is the subtraction that removes the trace from $3\boldsymbol{d} \otimes \boldsymbol{d}/r^8$; the separate `scalar` keeps the positive S/r^6 contact magnitude rather than a trace component.

3 The switching window

The switch reads $R_s = 4.3\,\text{Å}$ and $R_c = 5.0\,\text{Å}$ from the shared TOML file. The code returns full weight through the switch onset, zero weight at and beyond the cutoff, and the Brooks–CHARMM polynomial between them:

$$S(r) = \begin{cases} 1, & r \le R_s, \\ \dfrac{(R_c^2 - r^2)^2(R_c^2 + 2r^2 - 3R_s^2)}{(R_c^2 - R_s^2)^3}, & R_s < r < R_c, \\ 0, & r \ge R_c. \end{cases}$$

```
double DispersionSwitchingFunction(double r) {
    double r_switch = cfg("dispersion_switching_onset_distance"); // 4.3 A
    double r_cut    = cfg("dispersion_vertex_distance_cutoff");    // 5.0 A
    if (r <= r_switch) return 1.0;
    if (r >= r_cut)    return 0.0;

    double Rs2 = r_switch * r_switch;
    double Rc2 = r_cut * r_cut;
```

```
    double r2  = r * r;
    double num = (Rc2 - r2) * (Rc2 - r2) * (Rc2 + 2.0*r2 - 3.0*Rs2);
    double den = (Rc2 - Rs2) * (Rc2 - Rs2) * (Rc2 - Rs2);
    return num / den;
}
```

The endpoint values come from the factors in the middle branch. At $r = R_s$, the numerator equals the denominator, so $S = 1$. At $r = R_c$, the double factor $(R_c^2 - r^2)^2$ makes $S = 0$. Differentiating the middle branch gives

$$S'(r) = \frac{12r(R_c^2 - r^2)(R_s^2 - r^2)}{(R_c^2 - R_s^2)^3},$$

so $S'(R_s) = 0$ and $S'(R_c) = 0$. Since the outside branches are constant, the value and first derivative match at both joins. The code does not make curvature continuous; the second derivative of the middle branch is not constrained to equal the zero curvature of either constant branch.

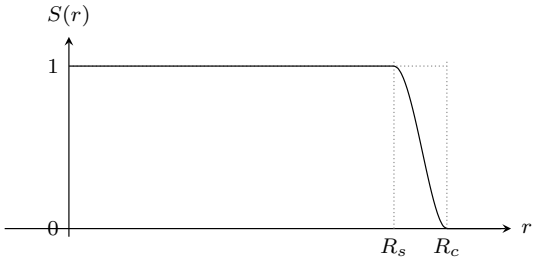

Figure 1: The switch is flat at full weight when it leaves the raw kernel at R_s, and flat again when it reaches zero at R_c. The plotted curve uses the configured switch for $R_s = 4.3$ Å and $R_c = 5.0$ Å.

With a hard cutoff, a continuously moving vertex that crosses R_c would change the scalar term from approximately R_c^{-6} to zero in one frame step, and the tensor term from $R_c^{-6}(3\hat{\boldsymbol{d}}\otimes\hat{\boldsymbol{d}} - \boldsymbol{I})$ to zero. The per-frame sum would inherit that finite jump from whichever vertex crossed. With this window, the switched scalar S/r^6 and switched tensor $Sr^{-6}(3\hat{\boldsymbol{d}}\otimes\hat{\boldsymbol{d}} - \boldsymbol{I})$ are already zero at R_c, and their first derivatives with respect to r are zero there. At R_s, $S = 1$ and $S' = 0$, so the derivative of the switched kernel matches the derivative of the raw kernel as the taper begins. The window removes value and slope discontinuities at the two switch boundaries; it does not remove a curvature change.

The C^1 property belongs to `DispersionSwitchingFunction`. The emitted kernel adds one more gate: `ComputeDispVertex` drops any vertex whose switch value falls below 10^{-15}, so near R_c, where S has already fallen below that floor, the kernel is set to zero rather than its taper value. The comparison is strict, so a value exactly at the floor is kept.

4 Accumulation and tensor split

After at least one vertex survives, the ring tensor, scalar contact, and contact count are stored on the ring-neighbour record. The tensor is also added to the atom total. Per ring type, the code accumulates the scalar contact sum and only the five T_2 components of the ring tensor. The per-type arrays hold `kAromaticRingTypeCount = 8` slots, indices 0 through 7; the proline pyrrolidine, type index 8, is the first saturated ring and falls outside the `ti < kAromaticRingTypeCount` guard, so its dispersion never reaches a per-type slot.

```
if (contacts == 0) continue;

ring_neighbour->disp_tensor = K_ring;
ring_neighbour->disp_spherical = SphericalTensor::Decompose(K_ring);
```

```
ring_neighbour->disp_scalar = s_ring;
ring_neighbour->disp_contacts = contacts;

int ti = ring.TypeIndexAsInt();
if (ti >= 0 && ti < kAromaticRingTypeCount) {
    ca.per_type_disp_scalar_sum[ti] += s_ring;
    for (int c = 0; c < kT2Components; ++c)
        ca.per_type_disp_T2_sum[ti][c] += ring_neighbour->disp_spherical.T2[c];
}

disp_total += K_ring;
ca.disp_shielding_contribution =
    SphericalTensor::Decompose(disp_total);
```

The exact arithmetic would keep every accepted $\boldsymbol{K}$ in T_2 only. The code does not symmetrize or remove the trace after summing, so any floating residue in T_0 or T_1 remains in `disp_shielding.npy`. The per-type T_2 array writes only the five stored shape components. The file `disp_per_type_T0.npy` writes per-atom, per-aromatic-type sums of s_{ring}, not the tensor trace average.

The shared projection is closed-form arithmetic on the 3×3 tensor `sigma` (here the ring or atom-total dispersion tensor). The trace divided by three gives T_0, the antisymmetric part gives T_1, and the symmetric traceless part gives the five T_2 numbers.

```
st.T0 = sigma.trace() / 3.0;

st.T1[0] = 0.5*(sigma(1,2) - sigma(2,1));
st.T1[1] = 0.5*(sigma(2,0) - sigma(0,2));
st.T1[2] = 0.5*(sigma(0,1) - sigma(1,0));

double Sxx = sigma(0,0) - st.T0;
double Syy = sigma(1,1) - st.T0;
double Szz = sigma(2,2) - st.T0;
double Sxy = 0.5*(sigma(0,1) + sigma(1,0));
double Sxz = 0.5*(sigma(0,2) + sigma(2,0));
double Syz = 0.5*(sigma(1,2) + sigma(2,1));

st.T2[0] = std::sqrt(2.0) * Sxy;
st.T2[1] = std::sqrt(2.0) * Syz;
st.T2[2] = std::sqrt(3.0 / 2.0) * Szz;
st.T2[3] = std::sqrt(2.0) * Sxz;
st.T2[4] = (Sxx - Syy) / std::sqrt(2.0);
```

The T_2 packing is isometric. The $\sqrt{2}$ factors on off-diagonal entries account for the Frobenius double count, since S_{xy} appears as both S_{xy} and S_{yx}. The two diagonal entries $\sqrt{3/2}\,S_{zz}$ and $(S_{xx} - S_{yy})/\sqrt{2}$ carry the remaining two degrees of freedom after $S_{xx} + S_{yy} + S_{zz} = 0$. A dot product of two stored T_2 five-vectors is therefore the Frobenius inner product of their symmetric traceless matrices.

5 Glossary

London dispersion kernel. The per-vertex tensor is a signed ellipsoid that stretches along the vertex-to-atom direction and shrinks equally in the two transverse directions. One accepted vertex at distance r contributes eigenvalues $2S/r^6, -S/r^6, -S/r^6$, while its separate scalar contact is S/r^6. Formally, $K_{ab} = S(r)r^{-6}(3\hat{d}_a\hat{d}_b - \delta_{ab})$, a symmetric traceless second-order tensor with units Å^{-6}.

C^1 **switching window.** The switch is a taper that leaves the kernel unchanged through R_s, lowers it across the cutoff interval, and reaches zero with a horizontal tangent. In this calculator $R_s = 4.3\,\text{Å}$, $R_c = 5.0\,\text{Å}$, and a vertex at the exact cutoff is removed because $S(R_c) = 0$ is below the numerical floor. Formally, the piecewise function has continuous value and first derivative at R_s and R_c, with no condition imposed on the second derivative.

Dyadic product. The grid a single direction builds by multiplying each of its components against every other (so one vector sets both rows and columns, and the matrix carries that one direction). For a vertex-to-atom unit direction, $\hat{\boldsymbol{d}} \otimes \hat{\boldsymbol{d}}$ has entry $\hat{d}_a \hat{d}_b$. Formally, for column vectors $\boldsymbol{u}, \boldsymbol{v}$, $(\boldsymbol{u} \otimes \boldsymbol{v})_{ab} = u_a v_b$.

Spherical tensor split. A 3×3 tensor holds three kinds of content that rotate without mixing — an orientation-free size (the scalar trace, T_0), an axial twist from its antisymmetric part (the three T_1 components), and a five-number traceless shape for the rest of the direction-dependence (the five T_2 components). In exact arithmetic, dispersion's tensor sum lives in the five T_2 components, while its positive $1/r^6$ contact sum is stored separately. Formally, it is the closed-form split of a 3×3 tensor's nine components into dimensions $1 + 3 + 5$, with the T_2 basis normalized to preserve the Frobenius norm.